# Plano técnico — confirmar à Meta na hora e responder em seguida

Nina / WhatsApp · 17/10/2026 · etapa 2 do plano de testes de carga · para aprovação antes de qualquer código

> **Objetivo:** o webhook do WhatsApp passa a responder "ok" à Meta em ~1–3 s (texto), em vez de esperar a Nina (~20 s, até 2 min). A resposta da Nina continua igual, só que gerada numa chamada separada, acionada pelo banco. Liga e desliga por clínica, sem publicar de novo.

## 1. Como é hoje

```
Meta → webhook: assinatura → grava a mensagem → portões* → Nina (agrupamento + IA + envio) → "ok" à Meta
                                                                  └── 20 s a 2 min ──┘
```

* portões: contato bloqueado, Francisco, confirmação de consulta, código do portal, pesquisa de satisfação, reabertura da conversa, regras de quem responde.

- A Meta reenvia a mensagem quando o "ok" demora: 2 de 24 mensagens reais chegaram duas vezes (~23 s depois).
- O reenvio encontra a conversa travada pela primeira execução → "conversa ocupada" → o webhook devolve 503 → a Meta reenvia de novo. Hoje **o reenvio da Meta é o mecanismo de recuperação** desses casos.
- Ponto a favor: a mensagem já é gravada antes da Nina, com chave única por mensagem, e a Nina já tem fila de turnos com vigia que retoma turnos travados.

## 2. Como fica

```
Meta → webhook: assinatura → grava → portões → [Nina deve responder?]
                                                  └ sim: registra a TAREFA e pede ao banco para acionar → "ok" à Meta (~1–3 s)

Banco (pg_net) → rota nova /api/public/nina/processar  (sem o limite de 30 s, a chamada fica aberta até 5 min)
                 token → assume a tarefa → relê mensagem, configuração e estado da conversa
                 → processarRespostaWhatsappNina(...)   ← a MESMA função de hoje (agrupamento, IA, envio)
                 → conclui a tarefa  |  "ocupada" → volta para a fila com espera curta

Varredura do banco (a cada 10 s): tarefa não iniciada em 60 s ou parada (reserva vencida) → aciona de novo
                 até 3 tentativas → depois encaminha para a equipe com o aviso padrão (nenhum paciente sem resposta)
```

**Por que pelo banco:** o Cloudflare corta trabalho feito depois do "ok" em 30 s, e ~1 em cada 10 respostas da Nina passa disso. Uma chamada aberta pelo banco é uma requisição nova, sem esse corte. O projeto já usa o mesmo mecanismo para o vigia (a cada minuto), backup e confirmação de consultas, e o executor de carga foi desenhado igual (`docs/nina/carga-servidor.md`).

## 3. O que muda no código

| PEÇA | O QUE FAZ |
|---|---|
| **Migration** `nina_whatsapp_assincrono` (aditiva, reexecutável, desligada ao aplicar) | Tabela `nina_whatsapp_tarefas`: uma linha por mensagem (única por `mensagem_id`), estado, token, tentativas, reserva (`expira_em`), último erro — sem texto nem telefone.<br>Configuração `nina_whatsapp_assincrono_config` (ativo, URL da rota com formato fixo) e token próprio em `sistema_job_tokens`.<br>Funções internas (SECURITY DEFINER, sem acesso para usuários): despachar, assumir, renovar, finalizar, varrer, diagnóstico.<br>Agendamento `nina-whatsapp-varredura` a cada 10 s (só consulta SQL; sem tarefa pendente, não faz chamada). |
| **Rota nova** `/api/public/nina/processar` | Exige o token interno (comparação de tempo constante) e aceita só `{tarefa, token}`. Todo o resto vem do banco. Renova a reserva a cada 20 s enquanto trabalha. |
| **Módulo novo** `whatsapp-assincrono.server.ts` | Despachar (usado pelo webhook) e executar a tarefa (usado pela rota). A montagem da entrada da Nina (texto, áudio que falhou, tipo, conversa) fica numa função única, usada pelos dois caminhos, para não divergir. |
| **Webhook** | Só o trecho `if (deveResponder)` muda: com a chave ligada e o despacho aceito, marca "despachado" e segue; com a chave desligada ou se o despacho falhar, faz exatamente o que faz hoje. |
| **Chave por clínica** | `nina_whatsapp_assincrono` em `clinica_feature_flags`. Desligar volta ao comportamento atual na hora, sem publicar. |

## 4. O que NÃO muda

- Toda a regra da Nina: agrupamento de mensagens seguidas, ordem, revisão (mensagem nova invalida resposta antiga), pedido médico, transferência, "Nina não responde conversa que está com gente", áudio de resposta, pesquisa.
- Os portões antes da Nina (bloqueio, Francisco, confirmação, portal, pesquisa, reabertura) continuam no webhook, na mesma ordem.
- Ambiente de testes e console de homologação (não passam pelo webhook).

> **Fica para uma fase 2 (se a medição pedir):** download de mídia, transcrição de áudio e leitura de foto continuam dentro do webhook, antes do "ok". Para texto, o "ok" sai em ~1–3 s; para áudio e foto, em ~5–15 s — ainda bem abaixo dos ~20 s em que a Meta reenviou.

## 5. Falhas e recuperação

| SITUAÇÃO | O QUE ACONTECE |
|---|---|
| Meta reenvia a mesma mensagem | Mensagem já gravada + tarefa única por mensagem → ignorada. |
| Banco não consegue registrar a tarefa | O webhook processa como hoje (nada se perde). |
| A chamada do banco não chega / servidor cai antes de assumir | Varredura aciona de novo depois de 60 s. |

| | |
|---|---|
| Servidor cai no meio da resposta | Reserva vence (90 s) → varredura aciona de novo; os pontos de controle do vigia de turno já existentes evitam mandar a resposta duas vezes. |
| "Conversa ocupada" (hoje: 503 para a Meta) | A tarefa volta para a fila com espera crescente (2 → 30 s). |
| 3 tentativas sem sucesso | Encaminha para a equipe com o aviso padrão, igual à desistência do vigia. |
| Configuração desligada | Despacho recusado → webhook processa como hoje. |

## 6. Segurança e LGPD

- A chamada do banco leva só o id da tarefa e um token aleatório da tarefa; texto, telefone e nome nunca saem do banco nessa chamada.
- Rota pública protegida por token guardado só no banco; funções internas sem acesso para `anon` / `authenticated`; tabela com RLS ligado e sem políticas (só o servidor).
- Logs só com ids, tempos e estados.

## 7. Testes automatizados

- Webhook com a chave desligada: todos os testes atuais de simulação do webhook passam sem mudança.
- Webhook com a chave ligada: responde "ok" sem chamar a Nina e registra uma tarefa; reenvio não cria segunda tarefa; falha no despacho cai no caminho de hoje.
- Rota: recusa sem token, com token errado ou com campo a mais; tarefa assumida uma vez só; entrada da Nina idêntica à que o webhook montaria (texto, áudio, foto, lista interativa).
- Recuperação: "ocupada" reagenda; reserva vencida volta; 3 falhas encaminham para a equipe.
- Migration aplicada duas vezes em PostgreSQL descartável (como no executor de carga), conferindo fila, varredura e permissões.

## 8. Implantação

| # | PASSO | QUEM |
|---|---|---|
| 1 | Branch com código, testes e migration (configuração desligada) → PR | Claude |
| 2 | Merge, publicação no Lovable e aplicação da migration | Você autoriza / aplica |
| 3 | **Prova da chamada longa:** ativar a configuração e rodar o diagnóstico (o banco chama a rota, que espera 90 s e registra o fim). Confirma que chamadas acima de 30 s terminam. | Você roda 1 comando SQL; Claude confere |
| 4 | Ligar a chave na Menino Jesus (única clínica com WhatsApp; volume real ainda baixo) e mandar do celular da equipe: texto, 3 seguidas, áudio, foto do pedido, pedido de atendente | Você / equipe |
| 5 | Conferir: cada mensagem entregue uma vez, "ok" em até 3 s para texto, tempo da Nina igual ao atual, nenhuma tarefa reacionada | Claude |
| 6 | Acompanhar 2–3 dias de tráfego real; depois seguir para a carga longa e o simulado com a equipe | Claude |

**Volta atrás:** desligar a chave da clínica (efeito imediato). Em último caso, desligar a configuração. Nada é apagado.

## 9. Decisões para você

1. Aprovar o desenho (despacho pelo banco + rota nova + varredura).
2. Autorizar a criação da migration (aditiva, desligada ao aplicar).
3. Ligar direto na Menino Jesus, atrás da chave, depois da prova do passo 3.
4. Deixar mídia (áudio e foto) para a fase 2.